

Overtune 3: Architecture, Wiki & Contributor Guide

DSP ENGINE INTERNALS, NUMERICAL STABILITY GUIDELINES, AND DEVELOPMENT PROTOCOL

1. Project Architecture & Repository Layout

Overtune 3 is developed with a strict decoupling between the mathematical DSP synthesis engine (pure C++20) and the desktop presentation layer (Qt 6 QML and Modern C++ wrapper classes).

```
overtune3/  
|-- dsp/                <-- Pure C++20 signal processing library (no GUI dependencies)  
|   |-- include/overtune/ <-- Public headers (FilterDesigner, Biquad, ComplexPoly)  
|   +-- src/            <-- Mathematical implementations and code generators  
|-- app/                <-- Desktop GUI application (Qt 6.4+, QML, Quick Controls)  
|   |-- src/            <-- C++ QObject models (FilterEngine, ExportModel, ThemeManager)  
|   +-- qml/            <-- Declarative UI (Pages, Controls, Custom Canvas Plots)  
|-- latex/              <-- Authoritative documentation sources and compiled PDFs  
+-- tests/              <-- Numerical test suites verified against SciPy benchmarks
```

2. The Core DSP Engine (dsp/)

The synthesis pipeline is encapsulated within the `overtune::FilterDesigner` class:

- **Prototype Synthesizer:** Generates analog prototype poles and zeros for normalized low-pass prototypes in the continuous s -domain.
- **Frequency Transformation:** Performs LP-to-HP, LP-to-BP, or LP-to-BS conformal mapping.
- **Bilinear Transform ($s \rightarrow z$):** Maps poles and zeros to discrete coordinates using pre-warped angular frequencies $\Omega = 2f_s \tan(\omega/2)$.
- **SOS Pairing & Ordering:** Decomposes the high-order transfer function into Second-Order Sections (SOS). Complex conjugate pole pairs with the highest Q are paired with nearest transmission zeros and placed downstream to maximize dynamic range and minimize register overflow.

3. Implementing a New Filter Family

To contribute a new filter approximation (e.g., Papoulis 'L' filter, Legendre, or Halpern):

1. Define the analog prototype root-finding algorithm in `dsp/src/Prototypes.cpp`.
2. Return a `std::vector<std::complex<double>>` of normalized s -plane poles and zeros.
3. Register the prototype enum identifier in `dsp/include/overtune/Types.h`.
4. Expose the property binding in `app/src/FilterEngine.h` and add the corresponding selector option in `app/qml/pages/DesignPage.qml`.
5. Write a unit test in `tests/test_prototypes.cpp` verifying magnitude response error $\leq 10^{-5}$ relative to SciPy reference vectors.

4. Numerical Accuracy & Stability Standards

All contributions must adhere to the following numerical standards:

- **Double-Precision Computation:** All internal prototype polynomials, matrix inversions, and root-finding iterations must use IEEE 754 64-bit double precision (`double`).
- **SOS Decomposition:** High-order filters must never be exported as single high-degree rational transfer functions $B(z)/A(z)$ due to root ill-conditioning for $N > 6$.
- **Jury Stability Verification:** Every synthesized filter must pass $|p_k| < 1 - 10^{-7}$ for all poles before being emitted to the UI or code exporter.
- **Zero Heap Allocation in Audio Thread:** Processing callbacks in `processBlock()` must execute in strictly bounded deterministic time with zero memory allocations or system calls.

5. Canonical DSP Literature References

Author(s)	Title	Core Topic
A. V. Oppenheim & R. W. Schaffer	<i>Discrete-Time Signal Processing</i>	Canonical LTI theory, Bilinear transform
S. K. Mitra	<i>Digital Signal Processing</i>	Comprehensive filter realization structures
L. R. Rabiner & B. Gold	<i>Theory and Application of DSP</i>	Filter approximation algorithms
A. Antoniou	<i>Digital Signal Processing</i>	Elliptic rational filter synthesis
R. Bristow-Johnson	<i>Audio EQ Cookbook</i>	Biquad parametric EQ derivations

Table 1: Foundational reference literature for Overtune 3 developers.

6. Git Workflow & Contribution Protocol

- **Branching Model:** Create feature branches branching from `main` named `feature/<filter-name>` or `fix/<issue-number>`.
- **Formatting:** Format all C++ code with `clang-format -style=file` adhering to the repository `.clang-format`.
- **Continuous Integration:** Automated GitHub Actions build on Linux, macOS, and Windows with clang and gcc. All regression unit tests must pass.
- **Pull Requests:** PRs must include a clear mathematical description of any proposed filter topology, accompanied by comparison plots generated from the test suite.